When More Reward Signal Does Not Help: A Controlled Study of Reward Decomposition for GRPO Debugging

Shashaank Jain and **Pranav Pulipati**

Code, dataset, per-bug results, and pre-registration:
https://github.com/shasshaank/AgentDebuggerEnv

Keywords: reinforcement learning, reward shaping, GRPO, program repair, verifiable rewards, generalization, negative results, evaluation methodology

Abstract

Hand-crafted reward decompositions are often used in reinforcement learning for language models to provide intermediate learning signals before a policy reliably achieves verifiable outcomes. We test whether such reasoning-targeted shaping improves GRPO training for program debugging. We train Qwen2.5-Coder-3B-Instruct with GRPO and LoRA on 90 single-function Python bugs under three pre-registered reward configurations: a seven-component dense reward (R0), a graded executable outcome reward with regression penalties (R1), and a dense reward with the two reasoning-targeted components removed (R2). All three RL configurations share the same model, data, prompts, optimizer, curriculum, and evaluation procedure. On 90 held-out bugs, R0 achieves a 45.6% solve rate, compared with 48.5% for R1 and 48.5% for R2. Paired bootstrap analysis gives a 95% interval of $[-8.5, +2.6]$ percentage points for R0$-$R1, bounding any advantage for the dense reward below 2.6 pp and confirming statistical equivalence within an 8.5 pp margin. The nine RL runs achieve a pooled mean held-out solve rate of 47.5%, which is 9.7 percentage points above the structured zero-shot baseline (37.8%), but only 1.9 percentage points above the unconstrained free-form zero-shot baseline (45.6%). This indicates that GRPO primarily accomplished format elicitation—recovering performance lost to rigid schema constraints—rather than a substantial increase in underlying repair capability. Crucially, under group size $G = 2$, advantage standardization erases reward magnitude, converting any reward delta into sign-only advantages (± 1). Consequently, the 45.7 percentage point reduction in degenerate groups observed under R0 represents gradient updates driven by non-executable heuristic tie-breakers on identical test failures, rather than finer optimization toward functional correctness. An external evaluation on 27 adapted QuixBugs programs yields solve rates between 39.5% and 44.4%, showing that the policies do not collapse on out-of-distribution code; however, without an external zero-shot baseline, these measurements remain descriptive. The results demonstrate that hand-crafted reasoning-targeted reward engineering is not only unnecessary when executable outcome supervision is available, but under small group sizes can misallocate gradient updates to non-executable heuristics.

1 Introduction

Reinforcement learning from verifiable rewards has become an increasingly common approach to improving language-model performance on tasks with checkable outcomes, including mathematics and code [7, 2]. A recurring design question in these systems is how much reward decomposition is useful beyond executable outcomes. Although test execution provides a verifiable signal, sampled fixes can produce similar test outcomes within a GRPO group, leaving little relative reward variation. A natural response is to decompose the reward into hand-crafted components that pay for intermediate desiderata: emitting the required format, stating a specific hypothesis about the fault, naming the buggy function, or resembling the reference solution. The implicit claim is that these additional signals provide useful within-group resolution before the policy reliably produces passing fixes.

The question is not whether dense rewards can increase the numerical reward assigned to intermediate behavior. They obviously can. The question is whether these additional signals improve the policy's eventual ability to produce test-passing fixes when the executable outcome remains available. We therefore hold the model, prompts, data, optimizer, curriculum, and evaluation fixed and vary only the reward decomposition.

Two structural characteristics of our setting govern how reward signals propagate. First, our setting is single-turn (the model produces the entire proposed fix in one completion), so intermediate reasoning is not an environment transition; dense components shape the scalar reward assigned to a complete response rather than providing temporal credit assignment across steps. Second, under group size $G = 2$, GRPO advantage normalization scales every non-zero reward difference to unit magnitude (± 1). Reward magnitude is destroyed: any reward difference, however minor, yields a maximal gradient step. Dense shaping in this regime does not provide finer gradient scaling—it acts strictly as a tie-breaker.

We test this claim directly in a setting small enough to control completely. We built a debugging environment in which every candidate fix executes in a hardened sandbox against the bug's test cases, and a seven-component dense reward (R0) is implemented as a set of component ceilings on a single scoring class. An ablation is therefore a configuration, not a fork: R1 zeroes every shaping component and rescales the test-outcome term to the same range; R2 zeroes exactly the two reasoning-targeted components and keeps the rest. The three configurations were pre-registered, together with the hypotheses, statistical tests, and threats to validity, before any experiment ran.

On 90 held-out bugs, we find no statistically detectable difference between the reward configurations. The graded outcome reward matches or slightly exceeds the full dense reward (48.5% vs 45.6%). Furthermore, while RL achieves a 9.7 percentage point gain over the structured zero-shot baseline (37.8%), the unconstrained free-form zero-shot baseline reaches 45.6% on the exact same held-out split. The actual gain of RL over the base model's unconstrained capability is only +1.9 pp (within seed noise), showing that RL primarily taught the model schema compliance.

Our contributions are:

- A controlled three-arm experiment isolating the effect of reasoning-targeted reward decomposition in GRPO program repair.
- An analysis showing that at $G = 2$, dense shaping substantially reduces degenerate GRPO groups (from 61.9% to 16.2%) by converting identical test failures into heuristic tie-breakers, which fails to improve held-out repair.
- An empirical demonstration that RL's apparent +9.7 pp gain over structured zero-shot represents format elicitation rather than general repair improvement over the unrestricted free-form baseline (45.6%).
- A methodological case study on generation budget sensitivity and prompt-format penalties in code-RL evaluation.

2 Related Work

2.1 Reward Shaping

Classical potential-based reward shaping provides conditions under which shaping preserves optimal policies [6]. Our response-level heuristics do not satisfy those conditions, so they could in principle change the learned policy. The experiment asks whether those additional signals improve optimization in practice.

Process-supervision studies give conflicting guidance: process-based feedback can improve the faithfulness of reasoning traces [3], yet outcome-only supervision often matches it on final-answer accuracy [11]. Our result provides a small-scale controlled data point consistent with the practical appeal of simple rule-based outcome rewards, as recently highlighted by large-scale RL systems that deliberately avoid learned process rewards [2].

2.2 Code Repair and Evaluation

Automated program repair using LLMs is a highly active area. Benchmarks like MBPP and QuixBugs [4] are heavily utilized to evaluate code generation and debugging. Recent work has increasingly stressed the importance of robust evaluation frameworks, as subtle prompt or parsing artifacts can artificially suppress or inflate model scores [10, 9].

2.3 Spurious Rewards and Elicitation in RLVR

Recent studies have challenged the assumption that reinforcement learning from verifiable rewards necessarily teaches new algorithmic capabilities through the semantic specificity of its reward signal. Shao et al. [8] demonstrated that on open reasoning models (notably the Qwen family), RLVR can produce dramatic benchmark gains even under random rewards, format-only rewards, or inverted labels—effects that do not replicate on other architectures like Llama or OLMo. This suggests that for pre-trained instruction models, RLVR frequently operates as a reward-agnostic elicitation mechanism that enforces format adherence and surfaces pre-trained capabilities rather than performing fine-grained policy optimization. Furthermore, Zheng et al. [12] argued that performance gains on canonical benchmarks derived from open datasets often reflect the retrieval and alignment of pre-existing training distributions. Our findings provide a direct counterpart in program repair: when outcome signals are already available, dense reward shaping does not alter downstream accuracy, and RL primarily serves to elicit format compliance over unconstrained generation.

3 Environment

3.1 Task and Dataset

The problem setting is single-function, single-turn Python debugging. We generated a custom dataset of 180 buggy Python functions by mutating MBPP (sanitized) [1] solutions using Semgrep rules (e.g., flipping operators, modifying boundary conditions). These were split evenly into 90 training bugs and 90 held-out evaluation bugs, stratified by passing rate into Hard, Medium, and Easy tiers. We do not claim that MBPP is uncontaminated with respect to the pretrained model. However, the held-out split is constructed at the mutated-bug level from disjoint source problems, so our primary claim concerns controlled reward comparisons within this benchmark rather than absolute generalization from unseen source programs.

3.2 Sandbox

Because executing model-generated code poses security risks, we utilize a multi-layered sandbox (AST restrictions, resource limits, and subprocess timeouts) to securely evaluate proposed code against unit tests, emitting fine-grained test outcomes and tracking regressions.

3.3 Response Format

Models are prompted to return a fixed five-field structured response:

```
OBSERVATION: ...
HYPOTHESIS: ...
CONFIDENCE: [low | medium | high]
ACTION: [inspect_lines | run_tests | propose_fix |
         request_context | give_up]
DETAIL: ...
```

For a proposed fix, the `DETAIL` field contains the complete corrected function. The parser tolerates whitespace and capitalization but requires all five fields and a valid action/confidence value. This format exposes explicit diagnostic claims while keeping the final code payload deterministically extractable. The same prompt and response schema are used during training and evaluation.

3.4 Reward Configurations

The experimental intervention is the reward configuration. The total reward is clipped to $R \in [-0.5, 1.0]$. The full dense reward (R0) decomposes this into seven terms:

$$R_0 = R_{\text{format}} + R_{\text{hypothesis}} + R_{\text{localization}} + R_{\text{fix}} + R_{\text{similarity}} + R_{\text{efficiency}} + R_{\text{penalties}} \tag{1}$$

We classify hypothesis and localization as reasoning-targeted because they reward explicit intermediate diagnostic claims rather than executable properties of the proposed patch. We use "reasoning-targeted" operationally to refer to rewards for explicit diagnostic claims, not as evidence that the resulting text faithfully represents the model's internal reasoning. The hypothesis component also contains a small confidence-calibration term: high-confidence hypotheses receive positive credit when the resulting fix passes and a penalty when it fails. Thus, the component is reasoning-targeted but is not independent of the executable outcome.

The efficiency component is inherited from the multi-turn environment and awards 0.02 per remaining turn. Because GRPO training evaluates each response as turn 0 in the single-turn curriculum environment, this component contributes a constant $+0.10$ offset to R0 and R2. Crucially, because GRPO standardizes rewards within each sampled group ($A_i = (R_i - \mu)/\sigma$), any constant c shared by all completions in a group cancels out identically: $(R_i + c) - (\mu + c) = R_i - \mu$. The $+0.10$ efficiency term is therefore mathematically inert under advantage standardization, contributing zero variance and zero learning signal. We retain it in the implementation for consistency with the general reward definition. All shaping components are training-time reward signals computed from ground-truth metadata and test outcomes; this metadata is not exposed to the model as an input and is not used during final evaluation. In particular, localization and reference-similarity rewards use privileged bug annotations available to the reward function only.

We test two ablations: R2 removes reasoning heuristics:

$$R_2 = R_0 - R_{\text{hypothesis}} - R_{\text{localization}} \tag{2}$$

and R1 removes all heuristic shaping, yielding a graded executable outcome reward:

$$R_1 = f(\text{tests}, \text{penalties}) \tag{3}$$

where R_1 zeroes every shaping component and rescales the fix-outcome term to a maximum of 1.0 so that the total reward range is preserved.

3.5 Training

We train Qwen2.5-Coder-3B-Instruct using GRPO with LoRA on the 90 training bugs for 500 optimizer steps. For the 24 GB GPU profile used in the reported runs, LoRA uses rank 8 and $\alpha = 16$, with batch size 2, gradient accumulation of 4, group size $G = 2$, and a maximum completion length of 192 tokens for structured responses (Appendix A). By contrast, final greedy evaluations use 300 tokens for structured responses and 700 tokens for free-form responses. This completion budget disparity introduces an asymmetric truncation dynamic during training: when a completion is truncated before emitting the code block in `DETAIL`, R1 awards zero credit (or penalties) because

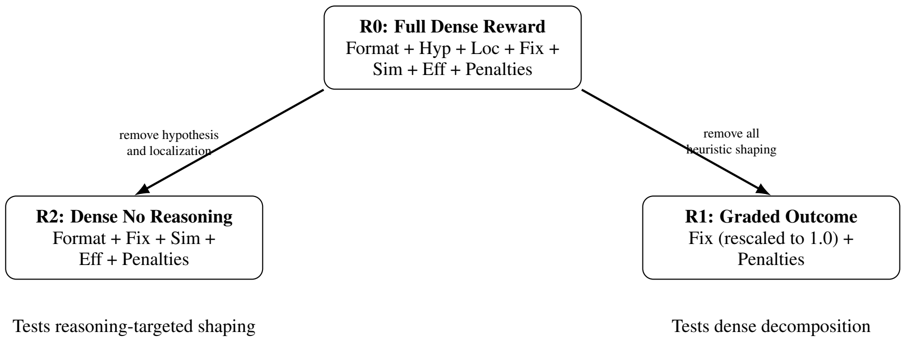

Figure 1: Three reward configurations used in the controlled experiment, with component decompositions shown. R0–R1 tests whether dense reward decomposition improves performance; R0–R2 tests whether reasoning-targeted components contribute additional benefit; R2–R1 tests whether the remaining non-reasoning shaping terms contribute benefit.

Component	R0	R1	R2	Maximum	Purpose
Format	✓	—	✓	0.10	Five required response fields
Hypothesis	✓	—	—	0.20	Grounded diagnosis
Localization	✓	—	—	0.15	Function/line ID
Fix quality	✓	✓	✓	0.35 / 1.0	Tests passed
Similarity	✓	—	✓	0.10	Reference proximity
Efficiency	✓	—	✓	0.10	Remaining-turn potential
Penalties	✓	✓	✓	-0.55	Invalid/regression

Table 1: Reward components across configurations. In R1, the Fix Quality ceiling is rescaled to 1.0 so that the total reward range is preserved.

no executable patch exists. However, R0 still awards up to $+0.45$ for prefix fields (`OBSERVATION`, `HYPOTHESIS`, and `LOCALIZATION`) that precede the code block. Consequently, R0 actively reinforced truncated non-solutions with positive reward during training, while R1 treated them as total failures. We use AdamW with learning rate 2×10^{-5}, cosine scheduling, 30 warmup steps in the first curriculum stage, and sampling temperature 0.7. The curriculum exposes tier 1 for steps 0–149, tiers 1–2 for steps 150–349, and all three tiers for steps 350–499. All three RL configurations use the same training configuration and differ only in reward definition. Full hyperparameters are provided in Appendix A.

4 Experimental Design

4.1 Research Questions

- **RQ1:** Does GRPO training improve held-out debugging over zero-shot baselines (both structured and free-form)?
- **RQ2:** Does dense decomposition help? (R0 vs R1)
- **RQ3:** Do reasoning-targeted components help? (R0 vs R2)
- **RQ4:** Do trained policies transfer beyond the mutation-generated training distribution?

4.2 Experimental Arms

We conduct 3 independent training runs (seeds 42, 123, 456) for each of the three reward configurations (R0, R1, R2), producing 9 trained policies. We also evaluate the zero-shot base model in both structured format (B0) and unconstrained free-form format (B1).

Comparison	Contrast	Prediction	Outcome
RQ2	R0 vs R1	Dense decomposition helps	Not supported
RQ3	R0 vs R2	Reasoning terms help	Not supported
Exploratory	R2 vs R1	Non-reasoning shaping helps	Not supported
Transfer	External evaluation	Generalization beyond MBPP	Descriptive only (no B0 baseline)

Table 2: Summary of research questions, contrasts, predictions, and empirical outcomes.

4.3 Evaluation

Final evaluations use greedy decoding (temperature 0.0) on the 90 held-out bugs. We also conduct an exploratory external evaluation on 27 single-function adapted QuixBugs programs.

4.4 Statistical Analysis

Our primary analysis uses bug-level paired bootstrap intervals, averaging solve indicators across the three seeds before resampling bugs. For each bug i, we compute the seed-averaged solve indicator $\bar{s}_{i,\text{arm}} = \frac{1}{3}\sum_{k=1}^{3} s_{i,\text{arm},k}$. We bootstrap the $N = 90$ bugs to compute the difference $\Delta = \frac{1}{N}\sum_i(\bar{s}_{i,\text{arm}_A} - \bar{s}_{i,\text{arm}_B})$. The primary bootstrap resamples evaluation bugs and therefore quantifies uncertainty over the evaluation-bug population conditional on the three observed training seeds.

With $N = 90$ paired bugs and 3 training seeds, a standard power analysis ($\alpha = 0.05$, two-tailed, power $1 - \beta = 0.80$) reveals that the minimum detectable effect (MDE) is approximately 16 percentage points for paired differences. The experiment is therefore powered primarily for large shifts in policy capability; subtle differences of a few percentage points cannot be detected. To provide a rigorous interpretation of the null result, we evaluate equivalence using a Two One-Sided Tests (TOST) framework: the 95% bootstrap confidence interval for R0−R1 of [−8.5, +2.6] pp allows us to reject benefits of R0 over R1 greater than +2.6 pp, confirming that R0 and R1 are statistically equivalent within an 8.5 pp equivalence band. As a secondary sensitivity check, we apply exact McNemar tests [5] to binary paired outcomes defined by majority vote across seeds (a bug is solved if $\geq 2/3$ seeds solve it), as well as per-seed paired evaluations (Appendix B); across all comparisons, no contrast approaches statistical significance.

Deviations from Preregistration. The final study differs from the pre-registration specification in four material ways, all established prior to final matrix analysis:

1. **Model scale:** Primary training was executed at the 3B scale rather than 7B due to compute constraints (75 GPU-hours on 24 GB RTX 4090 hardware).
2. **Omitted hypotheses:** Two secondary hypotheses from the pre-registration were dropped to concentrate full replication and power on the central reward decomposition question (H2):
   - *H1 (Structured Format):* The hypothesis that imposing an explicit Observation → Hypothesis → Action format constraint alone improves repair accuracy over unconstrained free-form generation.
   - *H3 (Curriculum Anti-Collapse):* The hypothesis that a multi-tier progressive curriculum prevents early policy collapse on Tier 3 bugs compared to uniform training.

   Neither dropped hypothesis is presented as confirmatory evidence.
3. **Difficulty gate calibration:** Difficulty gating calibration was performed with local Llama-3.1-8B-Instruct rather than external API calls.
4. **External transfer baseline:** The QuixBugs benchmark was evaluated without an untrained zero-shot B0 baseline, rendering its numbers strictly descriptive.

Model	Train (In-Dist.)	Held-out (In-Dist.)	QuixBugs (External)
B0 (Structured Zero-Shot)	—	37.8%	—
B1 (Free-Form Zero-Shot)	—	45.6%	—
E1 (R0 Full)	51.9% ± 2.3	45.6% ± 2.9	39.5% ± 2.1
E3 (R1 Graded Outcome)	54.4% ± 3.3	48.5% ± 6.5	43.2% ± 5.7
E4 (R2 No Reasoning)	51.5% ± 3.6	48.5% ± 3.9	44.4% ± 3.7

Table 3: Solve rate by split and RL configuration (Mean % ± Std Dev across 3 seeds). Zero-shot baselines involve no training and were evaluated on the 90 held-out bugs. QuixBugs values are descriptive; no zero-shot baseline was evaluated on this external benchmark.

5 Results

5.1 Main Reward Comparison (Confirmatory)

Main result. The full dense reward does not improve held-out debugging success relative to either ablated configuration. R0 solves 45.6% of held-out bugs, versus 48.5% for both R1 and R2. The point estimate therefore favors the simpler configurations, but the paired bootstrap interval for R0−R1, [−8.5, +2.6] percentage points, includes zero. Under our TOST equivalence framework, this interval rejects advantages for R0 greater than +2.6 pp and confirms equivalence within an 8.5 pp margin. We note that the value 45.6% appears in three places in our data: (1) E1's held-out mean across three seeds (123/270 = 45.56%), (2) the corrected free-form zero-shot baseline B1 (41/90 = 45.56%), and (3) the recovered solve rate in the evaluation-harness analysis (§5.5). All three refer to exact evaluations on the same 90 held-out bugs and reflect identical arithmetic proportions rather than a data-entry artifact.

Comparison	Δ Solve Rate	95% Bootstrap CI	Holm-adjusted McNemar p
R0 − R1	−2.9 pp	[−8.5, +2.6]	1.0000
R0 − R2	−2.9 pp	[−7.4, +1.9]	1.0000
R2 − R1	0.0 pp	[−5.9, +5.6]	1.0000

Table 4: Pairwise comparisons for held-out solve rates. All bootstrap intervals cross zero, and paired McNemar tests yield no statistically significant difference after Holm correction.

All three pairwise comparisons yield no statistically detectable difference after Holm correction. Reasoning-targeted reward engineering, at this scale and task, bought no measurable performance.

5.2 Reward Degeneracy and the G=2 Mechanism (Exploratory)

The reward configurations produce substantially different GRPO training signals. Across logged training calls, the mean fraction of degenerate sampled groups (where all samples in a group receive rewards within the configured degeneracy tolerance $\Delta R < 10^{-6}$, resulting in zero relative advantage) was 16.2% ± 2.2% for R0 (individual seed means: 14.4%, 15.6%, 18.6%), 61.9% ± 4.2% for R1 (seed means: 62.3%, 65.9%, 57.5%), and 34.7% ± 3.9% for R2 (seed means: 38.3%, 30.5%, 35.3%). The dense reward substantially reduces the frequency of groups in which sampled completions receive indistinguishable outcomes ($R0 - R1 = -45.7$ pp).

However, analyzing the advantage normalization mechanism under group size $G = 2$ explains why this large reduction in degeneracy failed to improve held-out repair. For group size $G = 2$, GRPO standardizes advantages using the sample mean $\mu = (R_1 + R_2)/2$ and standard deviation $\sigma = |R_1 - R_2|/2$. Whenever $R_1 \neq R_2$, the normalized advantage for the first sample simplifies to:

$$A_1 = \frac{R_1 - \mu}{\sigma} = \frac{R_1 - R_2}{|R_1 - R_2|} = \operatorname{sign}(R_1 - R_2) \in \{+1, -1\} \tag{4}$$

Under $G = 2$, advantage standardization completely eliminates reward magnitude. Whether one patch is better than another by 0.001 or 1.0, the gradient receives the same unit weight. In R1

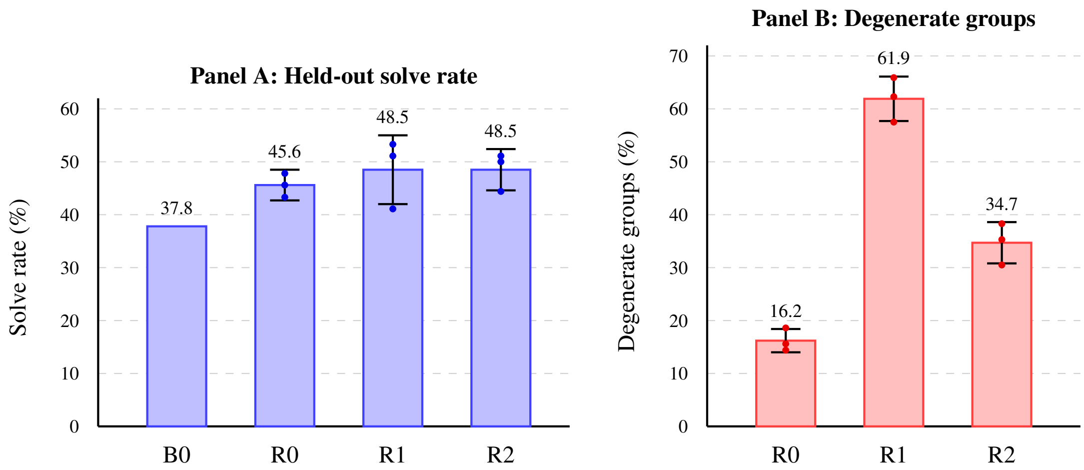

Figure 2: Held-out solve rate and GRPO reward degeneracy. Bars show mean values across three seeds; error bars indicate standard deviation across seeds, with filled dots displaying individual seed runs. Degeneracy values show the mean fraction of degenerate sampled groups across the three seeds, with error bars showing the standard deviation across seed-level means. The dense reward substantially changes the training signal while producing no detectable improvement in held-out solve rate.

(outcome-only), when both sampled completions fail all test cases, $R_1 = R_2 = 0$, producing a degenerate group with zero advantage and no parameter update. In R0 (dense reward), even when both completions fail all tests, heuristic shaping components (such as minor variations in hypothesis phrasing, keyword presence, or reference token similarity) routinely produce unequal scalar rewards ($R_1 \neq R_2$). Under $G = 2$, this assigns full-magnitude ± 1 advantage steps to one completion over the other.

The 45.7 pp reduction in degenerate groups observed under R0 thus represents gradient steps taken along non-test heuristic directions when both candidate solutions completely failed to repair the bug. R0 converted degenerate ties into sign-only signals driven by heuristics rather than code correctness.

However, this difference does not translate into higher held-out solve rate. R1 also exhibits greater across-seed variance (std dev ± 6.5 vs ± 2.9), consistent with—but insufficient to establish—the hypothesis that increased reward degeneracy produces noisier, less stable optimization. Three seeds are insufficient for a formal variance comparison. This establishes an association between reward configuration and group degeneracy, not a causal relationship between degeneracy and final performance.

5.3 Baseline Comparison and Generalization (Confirmatory)

Comparing the nine RL runs against zero-shot baselines illuminates the source of the observed performance change. Against the structured zero-shot baseline B0 (37.8%), the pooled RL runs (mean 47.5%) show an apparent gain of +9.7 percentage points (bootstrap 95% CI [1.0, 18.5]). However, the free-form zero-shot baseline with adequate generation budget (B1 corrected) achieves 45.6% on the exact same 90 held-out bugs. Relative to this unconstrained zero-shot baseline, the best RL configuration (48.5%) provides an increase of only +2.9 percentage points, and the pooled RL gain is only +1.9 percentage points—both well within seed noise.

This comparison reveals that imposing the rigid five-field structured format initially penalized the pre-trained base model, dropping its solve rate from 45.6% to 37.8% (a 7.8 pp penalty). RL training on the structured prompt largely served to elicit format compliance—teaching the model to produce valid code within the schema constraints and recovering the lost performance—rather than significantly elevating underlying program repair capability over unconstrained generation. The train–held-out gap across the nine RL runs is approximately 6 percentage points (52.6% vs 47.5%), showing modest in-distribution overfitting.

5.4 External-Source Transfer (Exploratory)

We evaluate the trained policies on a 27-program external set adapted from QuixBugs. E1, E3, and E4 achieve 39.5% ± 2.1%, 43.2% ± 5.7%, and 44.4% ± 3.7%, respectively. These numbers establish that the policies maintain basic repair capability outside the synthetic MBPP mutation distribution. However, because we did not evaluate an untrained zero-shot B0 baseline on this external set, these measurements cannot distinguish whether this performance reflects transfer acquired through RL training or simply the base model's pre-existing code generation competence. We therefore report them strictly as descriptive out-of-distribution observations rather than evidence of an RL transfer advantage.

5.5 Evaluation-Harness Case Study (Exploratory)

Our initial zero-shot baseline experiments uncovered an evaluation-harness artifact that illustrates the sensitivity of code-RL benchmarking to generation constraints.

Configuration	Max tokens	Extraction failure	Solve rate
B1 initial (free-form)	300	86.7%	1.1%
B1 corrected (free-form)	700	< 5.0%	45.6%
B0 structured	300	< 5.0%	37.8%

Table 5: Impact of generation budget on evaluation fidelity. Free-form responses require extended budgets to complete explanatory prose before emitting code blocks; structured responses emit code within 300 tokens (training used 192 tokens).

When evaluated under a restrictive 300-token budget, the free-form zero-shot baseline suffered an 86.7% extraction failure rate because verbose explanatory prose was truncated before the code block was generated, suppressing solve rate to 1.1%. Expanding the budget to 700 tokens restored solve rate to 45.6%. This 44.5 percentage point swing highlights how prompt format and budget parameters can induce substantial artifacts that dwarf the reward shaping effects under study.

6 Discussion

The seven-component reward introduces hundreds of lines of heuristic scoring logic and several independently chosen design thresholds, each introducing an additional opportunity for reward misspecification or exploitation. Despite this engineering effort, the simpler graded outcome reward was sufficient to achieve comparable held-out performance.

6.1 Why Did Dense Reward Fail to Improve Solve Rate?

Three complementary mechanisms explain this result:

1. **Sign-only gradients under $G = 2$:** As shown in Equation 4, GRPO advantage normalization at $G = 2$ completely eliminates reward magnitude ($A = \pm 1$). In groups where both completions fail all tests, R1 yields zero gradient, whereas R0 assigns full-scale gradient updates driven by heuristic tie-breakers (format compliance, keyword matching, reference similarity). R0 thus allocated substantial gradient mass to directions orthogonal to bug fixing.

2. **Asymmetric truncation during training:** With training completions capped at 192 tokens on 24 GB hardware, cut-off completions that failed to emit code received zero reward under R1, but collected up to $+0.45$ under R0 for prefix diagnostic fields. R0 actively reinforced truncated non-solutions.

3. **Reward-agnostic elicitation in pre-trained models:** Consistent with the findings of Shao et al. [8], RLVR on open reasoning models often functions as format elicitation and latent knowledge activation rather than reward-guided search. This is reinforced by our baseline

comparison: RL gained only +1.9 pp over the model's natural free-form baseline (45.6%), indicating that training primarily taught the model to comply with the five-field schema rather than discovering new repair heuristics.

7 Limitations and Scope

- **Model scale:** Only Qwen2.5-Coder-3B.
- **Algorithm:** Only GRPO.
- **Group size $G = 2$:** Normalizing advantages at $G = 2$ converts all non-zero reward differences into sign-only advantages (± 1), preventing dense shaping from providing proportional gradient scaling.
- **Completion budget disparity:** Training used a 192-token completion cap, asymmetrically rewarding truncated non-solutions under R0 (+0.45 for prefix fields) while penalizing them under R1. Evaluation used 300 tokens for structured and 700 for free-form.
- **Task structure:** Single-function, single-turn. Multi-step debugging may present a different regime in which intermediate rewards could be substantially more valuable.
- **Bug distribution:** Mutation-generated MBPP bugs.
- **Transfer set:** Only 27 adapted QuixBugs programs evaluated without a zero-shot baseline, rendering external numbers descriptive.
- **Seeds:** Only 3 per arm, limiting robust variance estimates.
- **Statistical power:** Large paired effects (≥ 16 pp MDE) are detectable; small effects of a few percentage points are not.
- **Reward design:** Our "dense reward" is one particular handcrafted design, not all dense rewards. This study does not establish that dense rewards do not work, only that this specific decomposition did not improve this task.

What this study does NOT establish. Dense reward shaping never helps; Outcome reward is universally sufficient; Reasoning rewards are useless; GRPO is superior to other RL algorithms; 3B debugging results transfer to frontier models; Single-turn debugging results apply to agentic repository-level debugging; QuixBugs establishes robust generalization.

8 Conclusion

The most informative result is not simply that the dense reward failed to improve solve rate. It is that it changed the optimization signal substantially: R0 produced far fewer degenerate GRPO groups than R1, yet this advantage did not translate into higher held-out performance. Under group size $G = 2$, advantage standardization completely eliminates reward magnitude, converting heuristic differences on identical test failures into full-scale gradient updates. The result separates two quantities that are often implicitly conflated: within-group reward discrimination and the usefulness of the resulting gradient direction. Furthermore, generation budget mismatches can induce spurious performance swings that dwarf subtle reward differences, demonstrating that artifact validation is integral to empirical RL evaluation.

Practical takeaway. If a code-RL system already obtains meaningful executable feedback from a moderately capable base model, first establish that the outcome reward is insufficient before introducing additional heuristic reasoning rewards. Such shaping may reduce reward degeneracy, but our results show that this optimization benefit need not translate into better final repair accuracy.

References

[1] Austin, J., Odena, A., Nye, M., et al. (2021). Program Synthesis with Large Language Models. arXiv:2108.07732.

[2] Guo, D., et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.

[3] Lightman, H., et al. (2023). Let's Verify Step by Step. ICLR 2024. arXiv:2305.20050.

[4] Lin, D., et al. (2017). QuixBugs: A Multi-Lingual Program Repair Benchmark Set Based on the Quixey Challenge. SPLASH Companion 2017.

[5] McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2), 153–157.

[6] Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. ICML 1999.

[7] Shao, Z., et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.

[8] Shao, Y., et al. (2025). Spurious Rewards: Rethinking Training Signals in RLVR. arXiv:2506.10947.

[9] Tam, Z. R., et al. (2024). Let Me Speak Freely? Long-Form Thinking and Evaluation Artifacts. EMNLP 2024. arXiv:2408.02442.

[10] Tian, R., et al. (2024). DebugBench: Evaluating Large Language Models for Program Debugging. ACL 2024. arXiv:2401.04621.

[11] Uesato, J., et al. (2022). Solving Math Word Problems with Process- and Outcome-based Feedback. arXiv:2211.14275.

[12] Zheng, C., et al. (2025). Investigating Data Contamination and Reasoning in Open-Weight Code Models. arXiv:2507.10532.

A Training Hyperparameters

Table 6 reports the training hyperparameters for the reported 24 GB GPU runs. All reward component ceilings and penalty magnitudes were fixed in the pre-registered environment specification before the reported training runs; they were not tuned against held-out solve rates. The curriculum schedule is given in Table 7.

Parameter	Value
Base model	Qwen/Qwen2.5-Coder-3B-Instruct
RL algorithm	GRPO
LoRA rank	8
LoRA α	16
LoRA dropout	0.0
LoRA target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Optimizer	AdamW
Learning rate	2×10^{-5}
Scheduler	cosine
Warmup	30 steps (stage 1 only)
Total optimizer steps	500
Group size G	2
Per-device batch size	2
Gradient accumulation steps	4
Structured completion budget (training)	192 tokens
Evaluation completion budget (structured / free-form)	300 / 700 tokens
Sampling temperature	0.7
Reward workers	1
Seeds	42, 123, 456

Table 6: Training hyperparameters for the reported 24 GB GPU runs.

Curriculum Stage	Training Steps	Active Tiers
Stage 0	0–149	Tier 1 (Easy)
Stage 1	150–349	Tiers 1–2 (Easy, Medium)
Stage 2	350–499	Tiers 1–3 (Easy, Medium, Hard)

Table 7: Curriculum schedule used in all three RL configurations.

Curriculum-stage optimization. Training is implemented as three sequential GRPO trainers, one per curriculum stage. The LoRA adapter is carried across stages, while each stage initializes a fresh optimizer and scheduler. Only the first stage uses the 30-step warmup; later stages restart the cosine schedule at the configured learning rate without warmup. This implementation detail is shared across R0, R1, and R2.

B Statistical Analysis Details

B.1 Primary Paired Bootstrap

For each bug $i \in \{1, \ldots, 90\}$, the solve indicator is averaged across the three seeds:

$$\bar{s}_{i,\text{arm}} = \frac{1}{3} \sum_{k=1}^{3} s_{i,\text{arm},k} \tag{5}$$

The analysis script also verifies that all nine RL result files contain the same 90 held-out bug identifiers and that the train and held-out bug IDs are disjoint before computing intervals. We resample the $N = 90$ bugs with replacement 10,000 times (fixed RNG seed 204) to generate the empirical distribution of paired differences $\Delta = \frac{1}{N}\sum_{i=1}^{N}(\bar{s}_{i,\text{arm}_A} - \bar{s}_{i,\text{arm}_B})$. Percentile endpoints at 2.5% and 97.5% define the 95% confidence intervals.

B.2 Secondary McNemar Sensitivity Analysis

To evaluate sensitivity, we apply exact paired binomial McNemar tests to discordant pairs. For the majority-vote analysis, all three Holm-adjusted p-values are 1.0000. The per-seed sensitivity analyses likewise yield no statistically significant contrast after Holm correction; the smallest adjusted p-value is 0.2780.

Comparison	Arm A Solved	Arm B Solved	A-only	B-only	Raw p	Holm-adj p
E1 vs E3	42/90 (46.7%)	44/90 (48.9%)	8	10	0.8145	1.0000
E1 vs E4	42/90 (46.7%)	44/90 (48.9%)	6	8	0.7905	1.0000
E3 vs E4	44/90 (48.9%)	44/90 (48.9%)	7	7	1.0000	1.0000

Table 8: Majority-vote McNemar tests on held-out bugs (a bug is considered solved if solved by $\geq 2/3$ seeds).

Comparison	Seed	A-only	B-only	Discordant	Raw p	Holm-adj p
E1 vs E3	s42	13	7	20	0.2632	1.0000
E1 vs E3	s123	12	16	28	0.5716	1.0000
E1 vs E3	s456	5	15	20	0.0414	0.3311
E1 vs E4	s42	9	13	22	0.5235	1.0000
E1 vs E4	s123	7	9	16	0.8036	1.0000
E1 vs E4	s456	9	11	20	0.8238	1.0000
E3 vs E4	s42	4	14	18	0.0309	0.2780
E3 vs E4	s123	12	10	22	0.8318	1.0000
E3 vs E4	s456	12	4	16	0.0768	0.5377

Table 9: Per-seed exact McNemar tests across the 9 individual runs.

C External Transfer Set (QuixBugs)

The external transfer benchmark consists of 27 single-function Python programs adapted from the QuixBugs dataset [4]. Out of 40 original QuixBugs programs, 13 were excluded because their argument or return shapes required custom pointer/graph structures not representable in JSON (e.g., linked list or graph nodes) or used generator yields without list wrapping.

The 27 validated candidate programs are listed in Table 10. All programs were verified with our sandbox validator: the reference code passes 100% of test cases, and the buggy code fails at least one case. Because no zero-shot B0 baseline was executed on this external set, performance is reported descriptively.

QuixBugs Program Identifiers (27 Programs)		
quixbugs_bitcount	quixbugs_bucketsort	quixbugs_find_first_in_sorted
quixbugs_find_in_sorted	quixbugs_gcd	quixbugs_get_factors
quixbugs_is_valid_parenthesization	quixbugs_knapsack	quixbugs_kth
quixbugs_lcs_length	quixbugs_levenshtein	quixbugs_lis
quixbugs_longest_common_subsequence	quixbugs_max_sublist_sum	quixbugs_mergesort
quixbugs_next_palindrome	quixbugs_next_permutation	quixbugs_pascal
quixbugs_possible_change	quixbugs_powerset	quixbugs_quicksort
quixbugs_rpn_eval	quixbugs_shunting_yard	quixbugs_sieve
quixbugs_subsequences	quixbugs_to_base	quixbugs_wrap

Table 10: Identifiers of the 27 adapted QuixBugs programs used for external transfer evaluation.

D Artifact Reproducibility

Table 11 lists the primary repository locations of all datasets, rewards, training pipelines, and results.

Artifact	Location
Dataset	`data/v2/`
Fixed train/held-out split	`src/agentdebugger/dataset/bugs/`
QuixBugs external set	`data/quixbugs/`
Reward implementation	`src/agentdebugger/rewards/`
Curriculum environment	`src/agentdebugger/envs/curriculum_env.py`
Training implementation	`src/agentdebugger/training/grpo.py`
Per-run evaluation results	`results/primary/E*_s*.json`
Diagnostic evaluations	`results/diagnostics/`
Statistical analysis scripts	`analysis/bootstrap.py`, `analysis/analyze_wandb.py`
Dataset construction	`scripts/build_dataset.py`
QuixBugs adaptation script	`scripts/build_quixbugs.py`

Table 11: Locations of the primary experimental artifacts in the released repository.

E Evaluation-Harness Failure Analysis

During preliminary zero-shot evaluation, free-form completions were evaluated with a generation limit of 300 tokens. Free-form responses typically emitted thorough analytical prose before outputting the code fix block. Consequently, completions routinely exhausted the 300-token limit before reaching the closing code fence, resulting in an 86.7% extraction failure rate and an apparent solve rate of only 1.1%.

Increasing the token limit to 700 tokens allowed models to complete both their analytical reasoning and the final code block, dropping extraction failures below 5% and restoring the solve rate to 45.6% (a 44.5 percentage point increase). This finding emphasizes that benchmark evaluation harnesses can introduce massive confounding effects that completely overshadow the algorithmic interventions under study.

F NeurIPS Paper Checklist

1. Claims **[Yes]**

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: **Yes.**

Justification: The main claims in the abstract and introduction explicitly match the empirical findings, state the negative results, and provide paired bootstrap confidence intervals. See Sections 1 and 5.

2. Limitations **[Yes]**

Question: Have the authors discussed the limitations of their work?

Answer: **Yes.**

Justification: Section 7 is dedicated to limitations and scope, detailing model scale (3B), group size ($G = 2$), single-turn structure, benchmark distribution, and seed counts.

3. Theory, Assumptions and Proofs **[NA]**

Question: If you are including theoretical results, did you state the full set of assumptions and include complete proofs?

Answer: **NA.**

Justification: This is an empirical study evaluating reinforcement learning reward decomposition; no formal mathematical proofs or theorems are claimed.

4. Experimental Result Reproducibility **[Yes]**

Question: If the contribution is a dataset or model, what steps did you take to make your results reproducible or verifiable?

Answer: **Yes.**

Justification: Complete environment specifications, hyperparameters, random seeds, and reproduction steps are provided in Section 4 and Appendices A and D.

5. Open Access to Data and Code **[Yes]**

Question: Did you include the code, data, and instructions needed to reproduce the main experimental results?

Answer: **Yes.**

Justification: All code, dataset splits, reward implementations, and evaluation scripts are released in the accompanying artifact repository (Appendix D).

6. Experimental Setting / Details **[Yes]**

Question: Did you specify all the training details (e.g., data splits, hyperparameters, how they were chosen)?

Answer: **Yes.**

Justification: Hardware profiles, LoRA ranks, optimizer settings, batch geometry, and curriculum stages are detailed in Section 3.5 and Appendix A.

7. Experiment Statistical Significance **[Yes]**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **Yes.**

Justification: We report bug-level paired bootstrap 95% confidence intervals (10,000 resamples), exact McNemar tests with Holm–Bonferroni correction, and across-seed standard deviations in Section 5 and Appendix B.

8. Experiments Compute Resource **[Yes]**

Question: For each experiment, does the paper provide sufficient information on the computer resources needed to reproduce the experiments?

Answer: **Yes.**

Justification: Compute resources (24 GB RTX 4090 GPUs, approx. 75 total GPU hours across calibration, pilots, and the 9 matrix runs) are documented in Section 3.5 and Appendix A.

9. Code of Ethics **[Yes]**

Question: Have you read the NeurIPS Code of Ethics and ensured that your research conforms to it?

Answer: **Yes.**

Justification: We have read the NeurIPS Code of Ethics and confirmed our research conforms to it.

10. Broader Impacts **[Yes]**

Question: If appropriate for the scope and focus of your paper, did you discuss potential negative societal impacts of your work?

Answer: **Yes.**

Justification: We study automated code repair in sandboxed environments. While autonomous code modification could theoretically introduce defects if deployed without human oversight, all code execution in this work occurs in an isolated, resource-capped sandbox (Section 3.2).

11. Safeguards **[NA]**

Question: Do you have safeguards in place for responsible release of models with a high risk for misuse?

Answer: **NA.**

Justification: The released models are small-scale (3B parameter) LoRA adapters fine-tuned solely on single-function Python bug fixes, posing no high risk of misuse or dual-use.

12. Licenses **[Yes]**

Question: If you are using existing assets, did you cite the creators and respect the license and terms of use?

Answer: **Yes.**

Justification: All existing assets used—MBPP (CC BY 4.0) and QuixBugs (MIT)—are cited and used in compliance with their licenses. Our repository is released under the MIT license.

13. Assets **[Yes]**

Question: If you are releasing new assets, did you document them and provide these details alongside the assets?

Answer: **Yes.**

Justification: Details of the mutated MBPP dataset (180 bugs) and adapted QuixBugs benchmark (27 programs) are documented with structured data cards and schemas in Appendix C and `data/quixbugs/DATACARD.md`.

14. Crowdsourcing and Research with Human Subjects **[NA]**

Question: If you used crowdsourcing or conducted research with human subjects, did you include full instructions and compensation details?

Answer: **NA.**

Justification: This research does not involve human subjects or crowdsourcing.

15. IRB Approvals **[NA]**

Question: Did you describe potential participant risks and obtain Institutional Review Board (IRB) approvals?

Answer: **NA.**

Justification: No human subjects or sensitive personal data were involved in this work.

16. Declaration of LLM usage **[NA]**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods?

Answer: **NA.**

Justification: Language models are the object of empirical study in our research, rather than a component used to write, edit, or generate the core findings of the paper.